Пермский институт (филиал) федерального государственного бюджетного
образовательного учреждения высшего образования
«Российский экономический университет имени Г.В. Плеханова»

Отделение информационных технологий и программирования

Практическая работа №2

Тема работы: “Разработка калькулятора на WPF”

Работу выполнил:
Щеглов Даниил Никитич
Группа: ИПо-31
Преподаватель: Берестов Дмитрий Борисович

Пермь 2026

СОДЕРЖАНИЕ

1. Введение: задачи практики 4

2. Что я разрабатывал 4

3. Организация проекта 5

4. Как устроен разбор выражения 6

4.1 Лексер: строка превращается в токены 6

4.2 Рекурсивный спуск вместо обратной польской записи 7

4.3 Приоритет операций и унарный минус 8

4.4 Вычисление на decimal 8

4.5 Понятные сообщения об ошибках 9

5. Интерфейс: MVVM, команды и журнал 10

5.1 Модель представления и команды 10

5.2 Живой предпросмотр 11

5.3 Журнал вычислений и хранение настроек 12

5.4 Оформление: темы и языки 13

6. Тестирование 14

6.1 Что и как я тестировал 14

6.2 Параметризованный тест приоритетов 14

6.3 Проверка ошибок 15

6.4 Тесты журнала и файлов 15

6.5 Результат прогона 16

7. Итоги выполнения 16

8. Чему я научился 17

9. Ссылки..... 18

1. Введение: задачи практики

В рамках практической работы я разработал настольное приложение-калькулятор на платформе WPF. Задачи, которые я ставил перед собой: освоить построение оконного приложения по паттерну MVVM; научиться разбирать арифметическое выражение, записанное одной строкой, с учётом приоритета операций и скобок; обеспечить точные вычисления без двоичной погрешности; сохранять журнал вычислений и настройки между запусками; покрыть вычислительное ядро модульными тестами.

Отличие этой работы от обычного калькулятора с двумя полями ввода состоит в том, что пользователь вводит всё выражение целиком, например 12+7*3 или (2+3)^2, а программа сама определяет порядок действий. Именно разбор выражения стал центральной частью работы.

2. Что я разрабатывал

Я разработал приложение «Калькулятор», которое принимает арифметическое выражение и вычисляет его значение. Поддерживаются четыре действия, возведение в степень и скобки любой вложенности. Выражение можно набирать как с экранной клавиатуры, так и с обычной: поле ввода принимает символы с клавиатуры, а кнопки добавляют очередной фрагмент в конец строки.

Помимо вычислений приложение ведёт журнал: каждое успешное вычисление попадает в список с датой и временем, а запись можно выбрать, чтобы вернуть выражение в поле ввода. Предусмотрены две темы оформления — тёмная и светлая, а также переключение языка интерфейса между русским и английским. Выбранная тема, язык и журнал сохраняются в файлы и восстанавливаются при следующем запуске.

Отдельно я продумал поведение при ошибках. Если выражение записано неверно, приложение не завершается и не показывает пустой экран: оно выводит сообщение с указанием места ошибки, например «Не закрыта скобка, открытая в позиции 1».

3. Организация проекта

Я создал решение ScheglovCalculator, в которое включил три проекта. Первый проект, Scheglov.Calculator.Core, содержит библиотеку классов с вычислительным ядром, журналом и настройками. Второй, Scheglov.Calculator.App, — оконное приложение WPF. Третий, Scheglov.Calculator.Core.Tests, — проект модульных тестов на базе MSTest.

Такое разделение даёт главное преимущество: ядро ничего не знает о WPF. В нём нет ни одного упоминания окон, кнопок и кистей, поэтому его можно тестировать обычными модульными тестами, которые работают за доли секунды и не требуют графической среды. Проект приложения зависит от ядра, а проект тестов — тоже только от ядра, но не от приложения.

Проект	Тип	Назначение
Scheglov.Calculator.Core	библиотека классов	лексер, разбор выражения, вычисление, журнал, настройки
Scheglov.Calculator.App	приложение WPF	окно, модель представления, команды, оформление
Scheglov.Calculator.Core.Tests	проект MSTest	29 модульных тестов на вычислительное ядро

Таблица 1 — Состав решения

Все проекты ориентированы на .NET 8: ядро и тесты — на net8.0, приложение — на net8.0-windows, поскольку WPF существует только в Windows. В тестовом проекте подключены пакеты Microsoft.NET.Test.Sdk версии 17.11.1, MSTest.TestAdapter и MSTest.TestFramework версии 3.6.4. Сторонних библиотек для MVVM я не использовал: реализация собственной

команды занимает около двадцати строк и лучше показывает, как устроена связка «кнопка — команда — метод».

4. Как устроен разбор выражения

Строка 12+7*3 сама по себе для компьютера бессмысленна: это последовательность символов. Чтобы получить результат, программа проходит три этапа: разбивает строку на токены, строит по токенам дерево выражения и вычисляет это дерево. Каждый этап вынесен в отдельный класс: Lexer, Parser и Evaluator.

4.1 Лексер: строка превращается в токены

Лексер читает строку слева направо и выдаёт последовательность токенов — чисел и знаков операций. Пробелы он пропускает, поэтому «3 * (1 + 3)» и «3*(1+3)» для него одинаковы. Число читается до первого символа, который уже не может быть его частью; разделителем дроби считается и точка, и запятая, так что результат не зависит от привычки пользователя.

```
if (char.IsDigit(symbol) || IsSeparator(symbol))
{
    index = ReadNumber(expression, index, tokens);
    continue;
}

var kind = symbol switch
{
    '+' => TokenKind.Plus,
    '-' or '\u2212' => TokenKind.Minus,
    '*' or '\u00D7' or '\u00B7' => TokenKind.Multiply,
    '/' or '\u00F7' or ':' => TokenKind.Divide,
    '^' => TokenKind.Power,
    '(' => TokenKind.OpenParen,
    ')' => TokenKind.CloseParen,
    _ => throw new CalculatorException($"Недопустимый символ «{symbol}».")
};
```

Отдельно я предусмотрел знаки, которые приходят с экранной клавиатуры: вместо звёздочки там рисуется ×, вместо косой черты — ÷. Если бы лексер понимал только ASCII-символы, кнопки работали бы иначе, чем

клавиатура. В конце списка токенов всегда добавляется признак конца выражения: благодаря ему разборщику не нужно отдельно проверять выход за границу массива.

4.2 Рекурсивный спуск вместо обратной польской записи

Для разбора я выбрал метод рекурсивного спуска. Его идея в том, что приоритет операций задаётся не таблицей и не переводом в постфиксную запись, а самой структурой методов: каждый следующий уровень вызывается из предыдущего и «связывает» свои операции крепче.

```
Выражение                                                    → Слагаемое
Слагаемое                      → Множитель { ("+" | "-") Множитель }
Множитель                        → Унарное { ("*" | "/") Унарное }
Унарное                                    → "-" Унарное | Степень
Степень                                  → Первичное [ "^" Унарное ]
Первичное       → Число | "(" Слагаемое ")"
```

Разбор суммы начинается с того, что разбирается первый множитель, а затем, пока встречаются плюс или минус, к нему присоединяются следующие множители. Получившиеся узлы становятся ветвями дерева: операция хранит левую и правую части, а не готовое число.

```
private                        ExpressionNode                        ParseAdditive()
{
    var                left                =                ParseMultiplicative();

    while     (Current.Kind     is     TokenKind.Plus     or     TokenKind.Minus)
    {
        var      operation      =      Advance().Kind      ==      TokenKind.Plus
            ?                                                    BinaryOperator.Add
            :                                                 BinaryOperator.Subtract;

        left   =   new   BinaryNode(operation,   left,   ParseMultiplicative());
    }

    return                                                                  left;
}
```

Цикл while, а не рекурсивный вызов для каждого следующего слагаемого, даёт левую ассоциативность: выражение 7-2-1 читается как (7-2)-1 и даёт 4, а не 6. То же правило действует для деления: 100/10/2 равно 5.

4.3 Приоритет операций и унарный минус

Самый интересный случай — степень. Она правоассоциативна, поэтому правая часть после знака ^ разбирается тем же методом, который умеет обрабатывать минус. Благодаря этому корректно работают оба выражения: 2^-2 даёт 0,25, а 2^3^2 было бы посчитано как 2^(3^2).

```
private ExpressionNode ParsePower()
{
    var left = ParsePrimary();

    if (Current.Kind != TokenKind.Power)
        return left;

    Advance();

    // Правая часть разбирается как унарное выражение, поэтому степень
    // правоассоциативна и допускает отрицательный показатель.
    return new BinaryNode(BinaryOperator.Power, left, ParseUnary());
}
```

Унарный минус стоит между умножением и степенью. Из-за этого выражение -2^2 трактуется как -(2^2) и даёт -4, а не 4. Это не случайность и не побочный эффект, а сознательно выбранное соглашение, которое совпадает с математической записью; оно зафиксировано отдельным тестом, чтобы будущая правка разборщика не изменила поведение незаметно.

4.4 Вычисление на decimal

Вычисление выполняет обход дерева: для числа возвращается его значение, для операции — результат применения операции к значениям двух ветвей. Все расчёты ведутся в типе decimal. Тип double хранит число в двоичном формате и не может точно представить 0,1, поэтому 0.1+0.2 в нём даёт 0.30000000000000004. В decimal мантисса хранится в десятичной системе, и тот же пример даёт ровно 0,3 — это проверяется тестом.

Арифметические операции обёрнуты в checked: если результат выходит за границы decimal, вместо тихого переполнения возникает исключение, которое затем превращается в понятное сообщение. Деление на ноль проверяется до

операции, так как для decimal это исключение DivisionByZeroException, а не бесконечность.

```
return                              node.Operator                                switch
{
    BinaryOperator.Add          =>          checked(left          +          right),
    BinaryOperator.Subtract         =>         checked(left         -         right),
    BinaryOperator.Multiply         =>         checked(left         *         right),
    BinaryOperator.Divide            =>            right            ==            0m
        ?   throw   new   CalculatorException("Деление   на   ноль   невозможно.")
        :                      left                      /                      right,
    BinaryOperator.Power             =>             Power(left,             right),
    _     =>     throw     new     CalculatorException("Неизвестная     операция.")
};
```

Степень реализована через алгоритм быстрого возведения: показатель степени раскладывается по битам, и число умножений получается порядка двоичного логарифма вместо линейного. Для школьных примеров это не имеет значения, но при 2^1000 разница заметна, а переполнение всё равно отслеживается.

```
while                       (exponent                       >                       0)
{
    if            ((exponent            &            1L)            ==            1L)
        result           =           checked(result           *           factor);

    exponent                                    >>=                                    1;

    if                       (exponent                       >                       0)
        factor           =           checked(factor           *           factor);
}
```

Показатель степени должен быть целым: выражение 9^0.5 не вычисляется, и приложение честно сообщает об этом, а не подставляет приближённый корень. Отрицательный показатель разрешён и даёт обратную величину; ноль в отрицательной степени запрещён отдельной проверкой.

4.5 Понятные сообщения об ошибках

Все ошибки разбора и вычисления собраны в одном типе CalculatorException. Сообщения содержат позицию в исходной строке, поэтому пользователь понимает, где именно он ошибся, а не только то, что выражение

неверно. Позиция хранится в каждом токене — для этого в структуре Token есть отдельное поле Position.

```
if (Current.Kind != TokenKind.CloseParen)
    throw new CalculatorException(
        $"Не закрыта скобка, открытая в позиции {opening + 1}.");

//                                                                    ...

if (parser.Current.Kind != TokenKind.End)
    throw new CalculatorException(
        $"Неожидаемый фрагмент «{parser.Current.Describe()}» " +
        $"в позиции {parser.Current.Position + 1}.");
```

Чтобы интерфейс не занимался перехватом исключений, я добавил метод TryEvaluate. Он возвращает объект CalculationResult с признаком успеха, значением и текстом ошибки. Модель представления просто смотрит на этот признак — и в коде окна не остаётся ни одного блока try/catch.

```
public static CalculationResult TryEvaluate(string expression)
{
    try
    {
        return CalculationResult.Success(Evaluate(expression));
    }
    catch (CalculatorException error)
    {
        return CalculationResult.Failure(error.Message);
    }
    catch (OverflowException)
    {
        return CalculationResult.Failure(
            "Результат выходит за допустимый диапазон чисел.");
    }
}
```

5. Интерфейс: MVVM, команды и журнал

5.1 Модель представления и команды

Окно приложения разделено на две части. Слева находится панель вычислений: поле выражения, строка результата, подсказка и кнопки. Справа — журнал вычислений. Разметка окна не содержит обработчиков нажатий: в файле MainWindow.xaml.cs остаётся только инициализация компонента, а вся логика находится в классе MainViewModel.

Связь кнопок с методами обеспечивает команда DelegateCommand, реализующая интерфейс ICommand. Она хранит два делегата: что выполнить и можно ли выполнить действие прямо сейчас. Например, кнопка «равно» неактивна, пока поле выражения пустое: команда вычисления получает предикат, проверяющий строку, и при каждом изменении выражения сообщает интерфейсу, что состояние изменилось.

```
public sealed class DelegateCommand : ICommand
{
    private readonly Action<object?> _execute;
    private readonly Predicate<object?>? _canExecute;

    public event EventHandler? CanExecuteChanged;

    public bool CanExecute(object? parameter) =>
        _canExecute?.Invoke(parameter) ?? true;

    public void Execute(object? parameter) => _execute(parameter);

    public void RaiseCanExecuteChanged() =>
        CanExecuteChanged?.Invoke(this, EventArgs.Empty);
}
```

Кнопки цифр и операций передают в команду параметр — строку, которую нужно дописать к выражению. Благодаря этому для всех двадцати кнопок используется один метод, а разметка остаётся короткой: в каждой кнопке указано только то, что она добавляет.

5.2 Живой предпросмотр

Пока пользователь набирает выражение, приложение пытается посчитать его «на лету» и показывает промежуточный результат серым цветом. Для этого в модели представления есть отдельный вызов Preview: он выполняет разбор, но не добавляет запись в журнал. Вычисление по кнопке «равно» вызывает уже Calculate, который журнал пополняет. Разделение важно: иначе журнал заполнялся бы промежуточными состояниями незаконченного выражения.

```
public CalculationResult Preview(string expression) =>
    Calculator.TryEvaluate(expression);

public CalculationResult Calculate(string expression)
```

```
{
    var           result           =           Calculator.TryEvaluate(expression);

    if                                                                (result.IsSuccess)
        _history.Add(new       HistoryEntry(expression.Trim(),       result.Value,
DateTime.UtcNow));

    return                                                                    result;
}
```

Кнопка CE убирает последнее введённое число, но оставляет знаки операций и скобки, а C очищает всё поле. Для CE я не стал хранить отдельную историю ввода: достаточно отступить от конца строки назад, пока встречаются цифры и разделитель дроби. Это короче и не расходится с тем, что видит пользователь.

5.3 Журнал вычислений и хранение настроек

Журнал отображается списком, в котором новые записи стоят сверху. Каждая строка показывает выражение, результат и время. Щелчок по строке возвращает выражение в поле ввода и сразу пересчитывает его — удобно, когда нужно повторить расчёт с небольшой правкой.

Хранится журнал в файле history.json в каталоге приложения в папке локальных данных пользователя. Количество записей ограничено сотней: при переполнении удаляются самые старые. Ограничение защищает файл от бесконечного роста, ведь калькулятором пользуются месяцами. Настройки лежат рядом, в файле settings.json.

Отдельно я предусмотрел устойчивость к повреждённому файлу. Если файл настроек окажется испорченным, приложение не падает, а берёт значения по умолчанию — это поведение также закреплено тестом. Запись выполняется через временный файл с последующей заменой, чтобы внезапное завершение работы не оставило половину документа.

5.4 Оформление: темы и языки

Цвета интерфейса собраны в одном объекте ThemePalette. Модель представления хранит ссылки на кисти и при переключении темы сообщает интерфейсу, что все цветовые свойства изменились; разметка ничего не знает о конкретных цветах. Добавление третьей темы сводится к описанию ещё одной палитры.

Тексты интерфейса вынесены в объект UiStrings с двумя наборами — русским и английским. Переключение языка меняет подписи кнопок, заголовки и подсказки, не пересоздавая окно. Выбранные тема и язык сохраняются в настройках, поэтому следующий запуск начинается в том же оформлении.

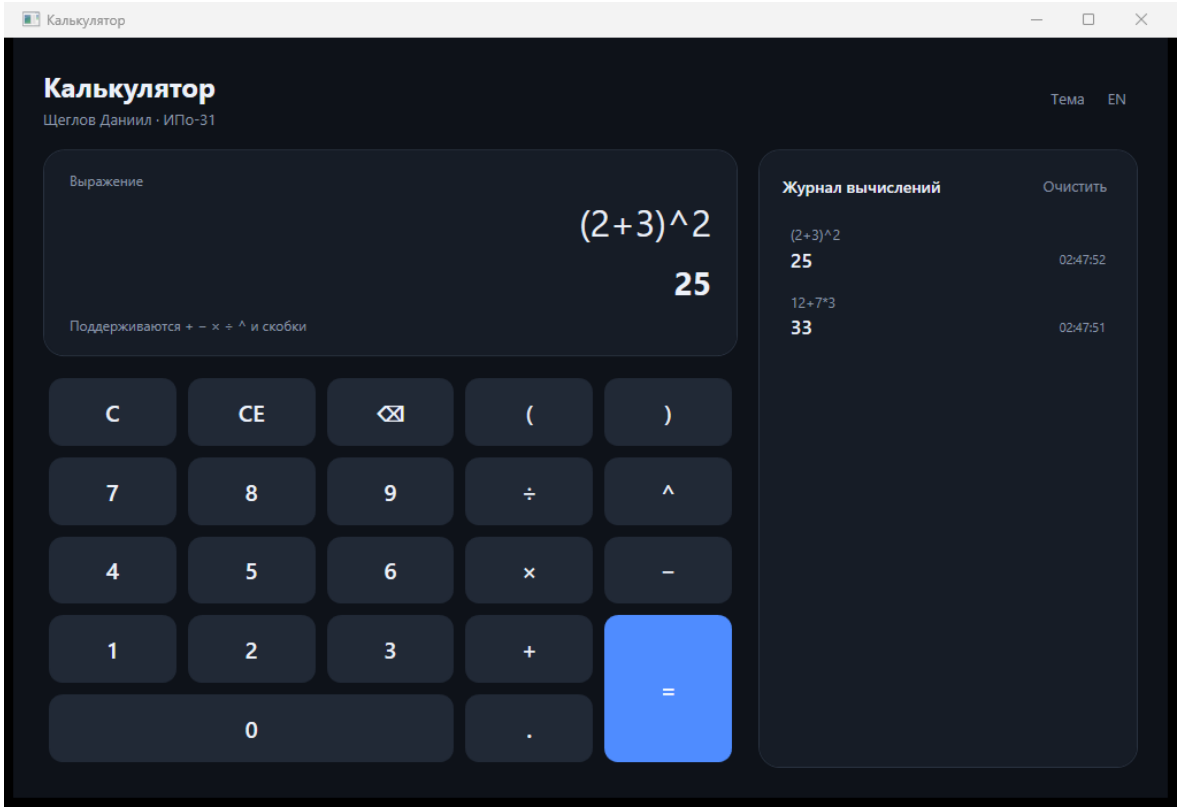


Рисунок 1 — Главное окно калькулятора: выражение (2+3)^2, результат 25 и журнал вычислений

6. Тестирование

6.1 Что и как я тестировал

Тестами покрыто вычислительное ядро: разбор выражений, вычисление, журнал и настройки. Интерфейс модульными тестами не покрывался, потому что в нём нет собственной логики — он только отображает состояние модели представления. Работу окна я проверил вручную: набрал (2+3)^2 и получил 25, набрал 12+7*3 и получил 33, убедился, что обе записи появились в журнале.

Каждый тест построен по схеме «подготовка — действие — проверка» и проверяет одно утверждение. Имена тестов описывают метод, условие и ожидаемое поведение, поэтому отчёт о прогоне читается как список требований к программе.

6.2 Параметризованный тест приоритетов

Проверки приоритета операций однотипны, поэтому я оформил их одним параметризованным тестом с набором строк и ожидаемых значений. Среди случаев есть и скобки, и степень с отрицательным показателем, и унарный минус, и цепочки одинаковых операций.

```
[DataTestMethod]
[DataRow("2+3*4",                                                          14)]
[DataRow("(2+3)*4",                                                        20)]
[DataRow("10/4",                                                          2.5)]
[DataRow("2^10",                                                         1024)]
[DataRow("2^-2",                                                         0.25)]
[DataRow("-2^2",                                                           -4)]
[DataRow("-(3+4)*2",                                                      -14)]
[DataRow("7-2-1",                                                           4)]
[DataRow("100/10/2",                                                        5)]
public    void    Evaluate_RespectsOperatorPrecedence(string    expression,    double
expected)
{
    var            result            =            Calculator.Evaluate(expression);

    Assert.AreEqual((decimal)expected,                                  result);
}
```

В атрибутах DataRow нельзя записать значение типа decimal: аргументы атрибутов допускают только простые типы, и компилятор приводит литерал к double. Поэтому ожидаемое значение объявлено как double, а внутри теста приводится к decimal. Если бы я сравнивал decimal с double напрямую, компилятор потребовал бы явного преобразования, а неявное сравнение могло бы дать неверный результат.

6.3 Проверка ошибок

Ошибочные выражения проверяются через Assert.ThrowsException. Этот метод сам фиксирует падение теста, если исключение не было выброшено, и возвращает перехваченный объект, что позволяет дополнительно проверить текст сообщения. Я проверяю не только тип исключения, но и то, что сообщение содержит нужное слово — иначе любая ошибка разбора проходила бы как «правильная».

```
[TestMethod]
public                  void                  Evaluate_UnclosedParenthesis_Throws()
{
    var         error         =         Assert.ThrowsException<CalculatorException>(
        ()                      =>                      Calculator.Evaluate("(2+3"));

    StringAssert.Contains(error.Message,                                  "скобка");
}
```

Отдельно проверены деление на ноль, пустое выражение, недопустимый символ, два числа без операции и дробный показатель степени. Кроме того, есть тесты на метод TryEvaluate: при ошибке он возвращает признак неудачи и текст, а при успехе — значение и пустое поле ошибки.

6.4 Тесты журнала и файлов

Тесты журнала работают с настоящими файлами, но во временном каталоге. Перед каждым тестом создаётся уникальная папка, после — удаляется. Такой приём позволяет проверить сохранение и загрузку, не затрагивая рабочие данные пользователя и не оставляя мусор на диске.

```
[TestInitialize]
public                              void                                CreateStorage()
{
    _storage                    =                   Path.Combine(Path.GetTempPath(),
        "calc-tests-"                +                Guid.NewGuid().ToString("N"));
    Directory.CreateDirectory(_storage);
}

[TestCleanup]
public                              void                                RemoveStorage()
{
    if                                                  (Directory.Exists(_storage))
        Directory.Delete(_storage,                recursive:                true);
}
```

Проверяется, что успешное вычисление попадает в журнал, что предпросмотр журнал не меняет, что неудачное выражение не сохраняется, что журнал восстанавливается после перезапуска и что при переполнении остаются только последние записи. Есть тест на испорченный файл настроек: он должен привести к значениям по умолчанию, а не к исключению.

6.5 Результат прогона

Набор содержит 29 тестов: 20 проверяют вычисления и разбор выражений, 9 — журнал и настройки. Все тесты проходят успешно, прогон занимает меньше секунды.

```
dotnet          test          ScheglovCalculator.sln          -c          Release

Пройден! : не пройдено 0, пройдено 29, пропущено 0, всего 29
```

7. Итоги выполнения

Разработано приложение-калькулятор, которое вычисляет выражения с учётом приоритета операций, скобок и степени. Вычислительное ядро отделено от интерфейса и покрыто 29 модульными тестами. Реализованы журнал вычислений, две темы оформления, два языка интерфейса и сохранение настроек между запусками.

Главный результат для меня — не само окно, а понимание того, как устроен разбор выражений. Рекурсивный спуск оказался короче и понятнее, чем

перевод в постфиксную запись: грамматика читается прямо в коде, а приоритеты операций видны по структуре методов. Кроме того, я убедился, что выбор типа данных влияет на результат: decimal избавил от погрешности, которая неизбежна при double.

8. Чему я научился

В ходе работы я закрепил следующие навыки:

– построение оконного приложения WPF с разметкой XAML и привязкой данных;

– разделение приложения на вычислительное ядро и интерфейс по паттерну MVVM;

– реализация собственной команды ICommand и управление доступностью кнопок;

– разбор арифметических выражений методом рекурсивного спуска;

– построение и обход дерева выражения;

– точные вычисления с типом decimal и контроль переполнения через checked;

– сериализация журнала и настроек в JSON с обработкой повреждённых файлов;

– написание модульных тестов MSTest, включая параметризованные тесты и проверку исключений;

– чтение отчёта о прогоне тестов и поиск причины падения.

При дальнейшем развитии работы имеет смысл добавить функции sin, cos и корень, поддержку переменных и памяти, а также тесты модели представления. Эти возможности не потребуют переписывания ядра: достаточно расширить грамматику и добавить новые узлы дерева.

9. Ссылки

1. Документация по WPF. — URL: https://learn.microsoft.com/ru-ru/dotnet/desktop/wpf/

2. Паттерн Model-View-ViewModel (MVVM). — URL: https://learn.microsoft.com/ru-ru/dotnet/architecture/maui/mvvm

3. Обзор языка C#. Тип decimal. — URL: https://learn.microsoft.com/ru-ru/dotnet/csharp/language-reference/builtin-types/floating-point-numeric-types

4. Модульное тестирование в .NET с помощью MSTest. — URL: https://learn.microsoft.com/ru-ru/dotnet/core/testing/unit-testing-with-mstest

5. Сериализация и десериализация JSON в .NET. — URL: https://learn.microsoft.com/ru-ru/dotnet/standard/serialization/system-text-json/overview